GYANMANJARI INNOVATIVE UNIVERSITY

Gyanmanjari Institute of Technology – Faculty of Engineering &Technology

Cloud-Based Software Engineering

Deploying a Flask Web Application on AWS Elastic Beanstalk

Student Name	Kothari Tarj Mehulbhai	Enrollment No.	260205020001
Subject	AdvanceSoftwareEngineering (METCE11501)	Semester	1 (M.Tech)
Activity	ALA 3	Marks	10

1. Aim

Todeployasimpleapplicationonacloudplatform(**AWS**),designitsarchitectureanddemonstratethelive deployment. The report includes the deployment steps, the architecture, screenshots, and a brief note on scalability and reliability.

2. Introduction

Cloud-basedsoftwareengineeringbuildsandrunssoftwareonon-demandinfrastructureinsteadofowned servers. AWS **Elastic Beanstalk** is a Platform-as-a-Service (PaaS): the developer uploads the code, and the service automatically provisions EC2 instances, a load balancer, auto scaling and monitoring.

Item	Choice
Cloud platform	Amazon Web Services (AWS), region ap-south-1 (Mumbai)
Service model	PaaS – AWS Elastic Beanstalk (Python 3.11 platform)
Application	Campus Notice Board – small Flask web app with /health endpoint
Tools	Python 3.11, Flask, Gunicorn, AWS CLI, EB CLI, Git (optional)

3. Application Description

Campus Notice Board shows a few notices and the name of the server (instance) that handled the request. Showing the instance name makes load balancing visible when more than one instance is running. A `/health`endpoint is used by the load balancer to check instance health.

4. Cloud Architecture

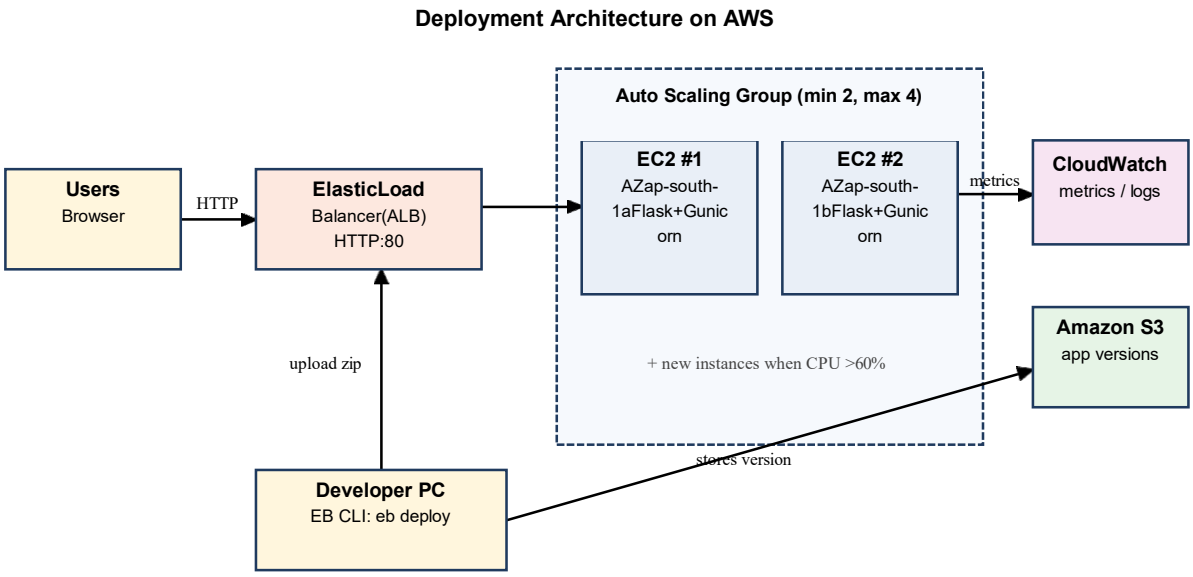


Figure 1: Architecture of the cloud deployment (ALB, Auto Scaling Group across two Availability Zones, S3, CloudWatch).

Component description

Component	Purpose
Application Load Balancer	Singlepublicentrypoint;distributesrequestsandremovesunhealthyinstances using /health.
EC2 instances (t3.micro)	Run the Flask app with Gunicorn. Placed in two Availability Zones.
Auto Scaling Group	Keeps at least 2 instances running; adds instances (up to 4) when CPU is high.
Amazon S3	Stores each uploaded application version (created by Elastic Beanstalk).
CloudWatch	Collects health, CPU and request metrics and application logs.
IAM roles	Give Elastic Beanstalk and the instances only the permissions they need.

5. Application Code

Project folder structure:

```
notice-board/
|--application.py             #ElasticBeanstalklooksfortheobjectnamed"application"
|--requirements.txt
`--.ebextensions/
    `--scaling.config         #autoscaling+healthchecksettings
```

application.py

```
importsocket
from datetime import datetime
from flask import Flask, jsonify

application=Flask(name)

NOTICES=[
    "Mid-semester exam timetable published",
    "Library open till 8 PM during exams",
    "SubmitActiveLearningActivityreportsontheGMIUWebPortal",
]


@application.route("/")
def home():
    items="".join(f"<li>{n}</li>"forninNOTICES)
```

```
    returnf"""
    <html><head><title>CampusNoticeBoard</title></head>
    <bodystyle="font-family:Arial;margin:40px">
      <h1>CampusNoticeBoard</h1>
      <ul>{items}</ul>
      <hr>
      <p>Servedbyinstance:<b>{socket.gethostname()}</b></p>
      <p>Time(server):{datetime.utcnow():%Y-%m-%d%H:%M:%S}UTC</p>
    </body></html>""
    "


@application.route("/health")
def health():
    returnjsonify(status="ok")


ifname=="main":
    application.run(host="0.0.0.0",port=8000)       #localtestingonly
```

requirements.txt

```
flask==3.0.3
gunicorn==22.0.0
```

.ebextensions/scaling.config

```
option_settings:
  aws:autoscaling:asg:
    MinSize:2
    MaxSize: 4
  aws:autoscaling:trigger:
    MeasureName:CPUUtilization
    Statistic: Average
    Unit: Percent
    UpperThreshold:60
    LowerThreshold: 20
  aws:elasticbeanstalk:environment:process:default:
    HealthCheckPath:/health
```

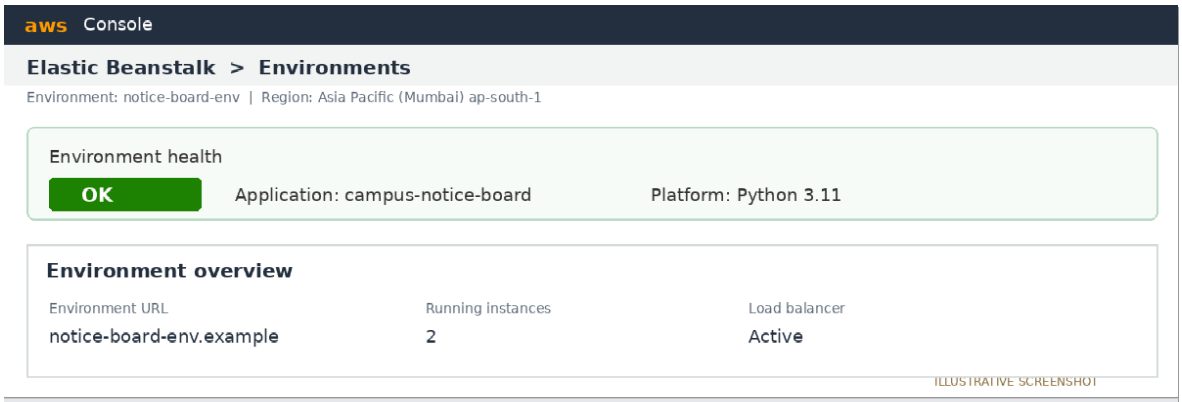
6. Deployment Steps

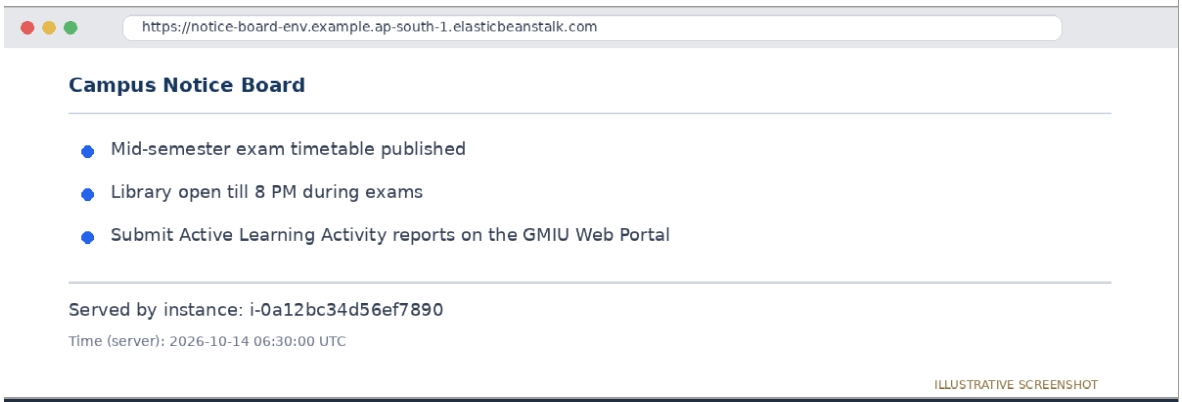
Step	Action	Command / Console action
1	Create an AWS account and an IAM user with programmaticaccess(avoidusingtherootaccount).	AWS Console →IAM →Users →Create user
2	Install and configure the AWS CLI.	`awsconfigure`(accesskey,secret,region ap-south-1)
3	Install the Elastic Beanstalk CLI.	`pip install awsebcli`
4	Test the app locally.	`pipinstall-rrequirements.txt;python application.py`; open http://localhost:8000
5	Initialise the Elastic Beanstalk application.	`eb init -p python-3.11 campus-notice-board --region ap-south-1`
6	Createtheenvironment(EC2,ALB,AutoScaling, security groups are created automatically).	`ebcreatenotice-board-env--elb-type application`
7	Check health and open the live site.	`eb status`; `eb open`
8	Verify the health endpoint and load balancing.	Refreshthepagerepeatedlyandwatchtheinstance name change.
9	Updatetheapp(e.g.,addanotice)andredeploy with zero downtime.	`eb deploy`
10	Clean up to avoid charges after the demonstration.	`eb terminate notice-board-env`

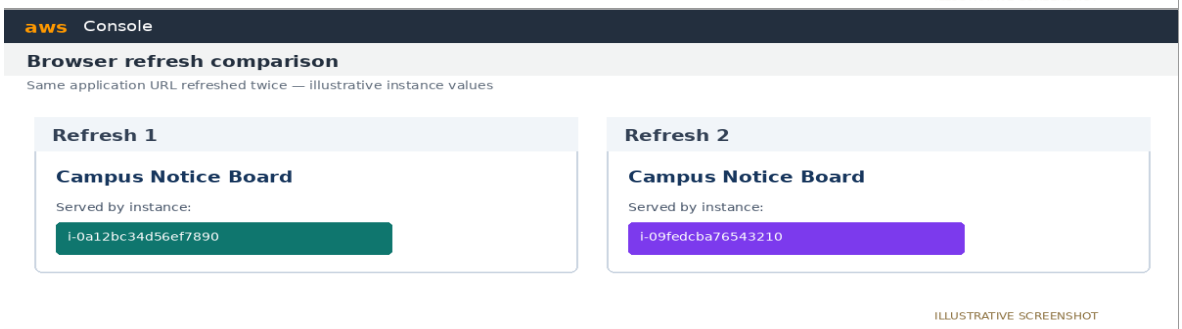
```
#summaryofthecommandsused
```

```
pipinstallawsebcli
eb init -p python-3.11 campus-notice-board --region ap-south-1
eb create notice-board-env --elb-type application
ebstatus
eb open
ebdeploy                          #afterchangingthecode
ebterminatenotice-board-env#removeallresourceswhenfinished
```

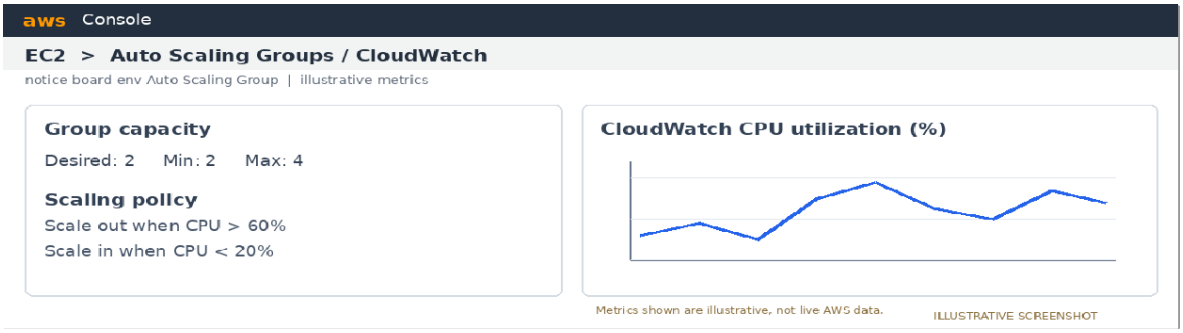
7. Live Deployment Demonstration











8. Scalability and Reliability

Scalability

- **Horizontal scaling:** the Auto Scaling Group adds EC2 instances (up to 4) when average CPU staysabove 60% and removes them below 20%.
- **Stateless design:** the application keeps no data on the server, so any instance can serve any request.
- **Load balancing:** the ALB spreads traffic evenly across all healthy instances.
- **Vertical scaling:** instance type can be changed (for example, t3.micro to t3.small) when more power per instance is required.

Reliability

- **Multi-AZ deployment:** instances in two Availability Zones keep the site running if one data centre fails.
- **Healthchecks:**theALBcalls/health;failinginstancesaretakenoutofrotationandreplaced automatically.
- **Rolling deployments:** `eb deploy`updates instances in batches so the site stays available.
- **Monitoring:** CloudWatch metrics and logs allow quick detection of problems.
- **Version history:** previous application versions in S3 allow quick rollback.

Limitations / possible improvements

- Add HTTPS with AWS Certificate Manager and a custom domain via Route 53.
- Store dynamic data in a managed database (Amazon RDS/DynamoDB) instead of a hard-coded list.
- Automate deployment with a CI/CD pipeline (GitHub Actions or AWS CodePipeline).

9. Conclusion

AFlaskapplicationwaspackagedanddeployedtoAWSElasticBeanstalk,whichcreateda load-balanced, auto-scaling environment across two Availability Zones. The activity showed how cloud platforms remove infrastructure effort and provide scalability and reliability through load balancing, health checks and automatic recovery.

References

- AWS Elastic Beanstalk Developer Guide – Python platform and EB CLI, docs.aws.amazon.com.
- Flask and Gunicorn documentation.

- IanSommerville,*SoftwareEngineering*,9thed.,PearsonEducation,2010(Cloud-basedsoftware engineering).